

JAVA DOCS

Design Patterns in Java

Creational, Structural & Behavioral Patterns with Practical Code

Java Agentic AI — Knowledge Base Reference

1. What Are Design Patterns?

Design patterns are reusable, proven solutions to common software design problems, first catalogued by the 'Gang of Four' (GoF) in 1994. They're grouped into three families: **Creational** (object creation mechanisms), **Structural** (composing classes/objects into larger structures), and **Behavioral** (communication and responsibility between objects).

2. Creational Patterns

2.1 Singleton

Ensures a class has exactly one instance, with a global access point.

```
public class ConfigManager {
    private static volatile ConfigManager instance;
    private ConfigManager() { }           // private constructor prevents external
instantiation

    public static ConfigManager getInstance() {
        if (instance == null) {
            synchronized (ConfigManager.class) {
                if (instance == null) {      // double-checked locking
                    instance = new ConfigManager();
                }
            }
        }
        return instance;
    }
}

// Cleanest modern approach - enum singleton (thread-safe, serialization-safe by default)
public enum Singleton {
    INSTANCE;
    public void doSomething() { }
}
```

2.2 Factory Method

```
interface Notification { void notifyUser(); }
class SMSNotification implements Notification {
    public void notifyUser() { System.out.println("SMS sent"); }
}
class EmailNotification implements Notification {
    public void notifyUser() { System.out.println("Email sent"); }
}

class NotificationFactory {
    public static Notification create(String type) {
        return switch (type) {
            case "SMS" -> new SMSNotification();
            case "EMAIL" -> new EmailNotification();
            default -> throw new IllegalArgumentException("Unknown type");
        };
    }
}
```

```
}  
}
```

2.3 Builder

Separates the construction of a complex object from its representation, especially useful when an object has many optional parameters (avoiding 'telescoping constructors').

```
public class Pizza {  
    private final String size;  
    private final boolean cheese;  
    private final boolean pepperoni;  
  
    private Pizza(Builder b) {  
        this.size = b.size; this.cheese = b.cheese; this.pepperoni = b.pepperoni;  
    }  
  
    public static class Builder {  
        private String size;  
        private boolean cheese, pepperoni;  
  
        public Builder size(String size) { this.size = size; return this; }  
        public Builder cheese(boolean v) { this.cheese = v; return this; }  
        public Builder pepperoni(boolean v) { this.pepperoni = v; return this; }  
        public Pizza build() { return new Pizza(this); }  
    }  
}  
  
Pizza pizza = new Pizza.Builder().size("Large").cheese(true).pepperoni(true).build();
```

Note: Lombok's `@Builder` and Java Records with a builder-style companion are common shortcuts in real codebases to avoid writing this boilerplate by hand.

2.4 Prototype

```
class Sheep implements Cloneable {  
    String name;  
    Sheep(String name) { this.name = name; }  
  
    @Override  
    public Sheep clone() {  
        try { return (Sheep) super.clone(); }  
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }  
    }  
}  
  
Sheep dolly = new Sheep("Dolly");  
Sheep clone = dolly.clone();
```

3. Structural Patterns

3.1 Adapter

Converts one interface into another that a client expects, without modifying either.

```

interface MediaPlayer { void play(String fileName); }

class LegacyPlayer { void playOldFormat(String file) { System.out.println("Playing " +
file); } }

class MediaAdapter implements MediaPlayer {
    private final LegacyPlayer legacyPlayer = new LegacyPlayer();
    public void play(String fileName) { legacyPlayer.playOldFormat(fileName); }
}

```

3.2 Decorator

Adds behavior to an object dynamically without modifying its class (composition over inheritance).

```

interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost() { return 2.0; } }

abstract class CoffeeDecorator implements Coffee {
    protected final Coffee wrapped;
    CoffeeDecorator(Coffee c) { this.wrapped = c; }
}

class MilkDecorator extends CoffeeDecorator {
    MilkDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 0.5; }
}

Coffee order = new MilkDecorator(new SimpleCoffee());
System.out.println(order.cost()); // 2.5

```

Note: *java.io is built almost entirely on Decorator: BufferedReader wraps a Reader, InputStreamReader wraps an InputStream, etc.*

3.3 Facade

Provides a simplified, unified interface to a complex subsystem, hiding internal complexity from the client (e.g. a single OrderFacade.placeOrder() that internally coordinates inventory, payment, and shipping services).

3.4 Proxy

```

interface Image { void display(); }

class RealImage implements Image {
    private final String file;
    RealImage(String file) { this.file = file; loadFromDisk(); }
    private void loadFromDisk() { System.out.println("Loading " + file); }
    public void display() { System.out.println("Displaying " + file); }
}

class ProxyImage implements Image {           // lazy-loading proxy
    private final String file;
    private RealImage real;
    ProxyImage(String file) { this.file = file; }
    public void display() {

```

```

        if (real == null) real = new RealImage(file); // load only when first needed
        real.display();
    }
}

```

Note: Spring AOP and Hibernate lazy-loaded entities are built on dynamic proxies -- this is why the Proxy pattern is a heavily interview-tested topic for Spring roles.

4. Behavioral Patterns

4.1 Strategy

Encapsulates interchangeable algorithms behind a common interface, selected at runtime.

```

interface PaymentStrategy { void pay(double amount); }
class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) { System.out.println("Paid " + amount + " via credit card"); }
}
class PayPalPayment implements PaymentStrategy {
    public void pay(double amount) { System.out.println("Paid " + amount + " via PayPal"); }
}

class Checkout {
    private PaymentStrategy strategy;
    Checkout(PaymentStrategy strategy) { this.strategy = strategy; }
    void process(double amount) { strategy.pay(amount); }
}

```

4.2 Observer

Defines a one-to-many dependency: when the subject changes state, all registered observers are notified.

```

interface Observer { void update(String event); }

class EventPublisher {
    private final List<Observer> observers = new ArrayList<>();
    void subscribe(Observer o) { observers.add(o); }
    void publish(String event) {
        for (Observer o : observers) o.update(event);
    }
}

```

Note: Java's own (now largely deprecated) `java.util.Observer/Observable` predates modern reactive libraries; today, Spring's `ApplicationEventPublisher` and reactive streams (Project Reactor) fill this role.

4.3 Template Method

```

abstract class DataProcessor {
    public final void process() {                // template - fixed algorithm skeleton
        readData();
        processData();
    }
}

```

```

        writeData();
    }
    abstract void readData();
    abstract void processData();
    void writeData() { System.out.println("Default write"); } // optional override hook
}

```

4.4 Command

```

interface Command { void execute(); }

class LightOnCommand implements Command {
    private final Light light;
    LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.turnOn(); }
}

class RemoteControl {
    private Command command;
    void setCommand(Command c) { this.command = c; }
    void pressButton() { command.execute(); }
}

```

4.5 Chain of Responsibility

Passes a request along a chain of handlers until one handles it — the basis of servlet filter chains and Spring Security's filter chain.

5. Pattern Selection Cheat Sheet

Problem	Pattern
Need exactly one instance globally	Singleton
Object creation logic should be decoupled from usage	Factory Method / Abstract Factory
Complex object with many optional parameters	Builder
Incompatible interfaces need to work together	Adapter
Add responsibilities to objects dynamically	Decorator
Control/delay access to an expensive object	Proxy
Interchangeable algorithms selected at runtime	Strategy
Notify many dependents of a state change	Observer
Fixed algorithm skeleton with customizable steps	Template Method

6. Quick Interview Recap

- Singleton vs static class? Singleton can implement interfaces, be passed as an object reference, support lazy initialization, and be subclassed/mockd more easily than a static utility class.
- Strategy vs State pattern? Structurally similar (both delegate to an interchangeable object), but Strategy is chosen explicitly by the client for an algorithm; State represents internal, self-transitioning object behavior tied to its current condition.
- Why enum singleton is preferred? It's inherently thread-safe, and the JVM guarantees serialization and reflection cannot create a second instance — unlike hand-rolled double-checked locking.
- Decorator vs Inheritance? Decorator adds behavior at runtime via composition to any combination of wrappers; inheritance is static and can lead to a combinatorial explosion of subclasses for every feature combination.